



SOFTWARE DESIGN DOCUMENT

VisionScan POS

Camera-based retail checkout system

Products are identified by an AI vision model rather than a barcode, matched against inventory, accumulated in a session, and converted into a bill at checkout.

VERSION

1.0

STATUS

Draft

PLATFORM

React · FastAPI · Supabase

What the system does, in one paragraph

VisionScan POS is a production-ready retail checkout application that uses computer vision to detect products during a shopping session. **Products are identified by an AI vision model — mocked in the initial release** — mapped to an inventory catalogue, accumulated in a customer session, and converted into a bill during checkout.

Inventory management

A catalogue of SKUs, prices, stock levels and aliases.

Stock deduction

Checkout reduces stock inside a single database transaction.

Session tracking

Each shopping trip is a session with its own scanned items.

Audit history

Every inventory movement is recorded with a reason code.

What version 1 is, and just as importantly, what it is not

In scope

- Inventory management
- Camera scan session management
- Product detection API
- Product matching engine
- Checkout engine
- Inventory updates
- REST APIs
- React administration UI

Out of scope for v1

- Payment gateway
- Authentication
- Multiple stores
- Loyalty programme
- Live camera integration
- Offline mode
- Receipt printing

The exclusions matter more than the inclusions — they are the reason this release can be estimated at all.

Three layers, one direction of travel



The first principle is the load-bearing one. It is what makes the database swappable and the API layer portable.

Six requirements carry the whole system

FR-001

Start scan session

Generate a session UUID, record check-in, set status ACTIVE.

FR-002

Scan product

Accept a detection, validate confidence, match, deduplicate.

FR-003

View session

Return every scanned line with quantity, price and confidence.

FR-004

End session

Mark the session finished. Inventory is deliberately untouched.

FR-005

Checkout

Validate stock, total, deduct, record movements — transactionally.

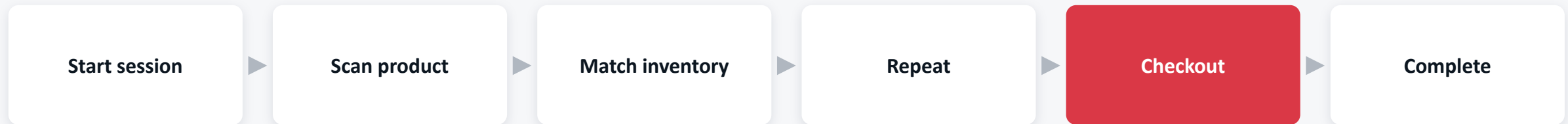
FR-006

Inventory management

Create, update and view items. Deletion is not supported.

FR-004 and FR-005 are separated on purpose: ending a session and paying for it are different events.

The session lifecycle



The decision inside every scan

Already in this session?

Yes

Increase quantity on the existing line

No

Insert a new scan item

What checkout does, in order

- Validate stock
- Deduct inventory
- Generate the bill
- Complete the session

Accepting a detection from the vision model

Input

```
{  
  detected_name,  
  confidence  
}
```

FR-003 returns

Product name · SKU · quantity ·
unit price · total · confidence

PROCESSING

- 1 Validate the confidence value
- 2 Match the product against inventory
- 3 Deduplicate within the session
- 4 Increment quantity if it already exists
- 5 Otherwise insert a new scan item

Deduplication is a requirement, not an optimisation — it is what stops one apple appearing four times.

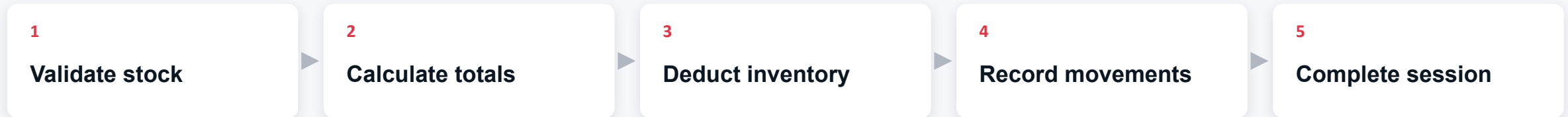
Three attempts, in strict priority order



Why the order is fixed

Fuzzy matching is the only stage that can be wrong in an interesting way. Putting it last means it only ever runs on names that exact and alias matching have already failed to resolve — which keeps the failure surface small and the behaviour reproducible.

The transactional core of the system



If any inventory validation fails, nothing happens at all

No inventory changes. No checkout. Full transaction rollback. There is no partially-completed checkout state in this design — a checkout either happened or it did not.

BR-001

A session may be checked out once, and only once.

BR-004

Checkout fails when stock is less than the requested quantity.

BR-006

The whole checkout runs inside a single database transaction.

Four tables

inventory_master

```
id UUID
sku TEXT
name TEXT
category TEXT
price NUMERIC
stock INTEGER
aliases TEXT[]
created_at TIMESTAMP
```

scan_sessions

```
id UUID
status ACTIVE /
COMPLETED
check_in_time
TIMESTAMP
check_out_time
TIMESTAMP
```

scan_items

```
id UUID
session_id UUID
inventory_id UUID
detected_name TEXT
confidence FLOAT
quantity INTEGER
first_seen TIMESTAMP
```

inventory_movements

```
id UUID
inventory_id UUID
change_qty INTEGER
reason SALE /
RESTOCK
created_at TIMESTAMP
```

inventory_movements is the audit table — every successful checkout writes to it, which is what makes stock history reconstructable.

Nine endpoints, four groups

Sessions

POST /sessions/start

GET /sessions/{id}

POST /sessions/{id}/end

Scan

POST /sessions/{id}/scan

GET /sessions/{id}/items

Inventory

GET /inventory

POST /inventory

PUT /inventory/{id}

Checkout

POST /sessions/{id}/checkout

Every one of these is called by React. None of them is bypassed — that is acceptance criterion AC-009.

The six statements the code must never violate

BR-001

A session may only be checked out once.

BR-002

Duplicate scans increase quantity instead of creating duplicate rows.

BR-003

Inventory matching priority is exact name, then alias, then fuzzy.

BR-004

Checkout fails when stock is less than the requested quantity.

BR-005

An inventory movement record must exist for every successful checkout.

BR-006

Checkout must execute within a single database transaction.

These are testable sentences. That is the whole point of writing them down before the build starts.

The constraints that shape the implementation

API RESPONSE TIME

< 300 ms

DATABASE

PostgreSQL

ARCHITECTURE

Layered

REST STANDARD

JSON

CODE COVERAGE

≥ 80%

DEPLOYMENT

Docker compatible

LOGGING

Structured

VALIDATION

Pydantic

DB TRANSACTIONS

Required

Every one of these is measurable. A non-functional requirement you cannot test is a preference.

One error shape, six mapped conditions

400 Validation error

404 Session not found

404 Product not found

409 Insufficient stock

409 Duplicate SKU

500 Internal error

Standard error envelope

```
{
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "Available stock is less
               than requested quantity.",
    "details": {}
  }
}
```

A machine-readable code, a human-readable message, and a slot for structured detail. Clients switch on the code, never on the message text.

Six controls, none of them optional

1 Validate every payload

Pydantic schemas on all request bodies. No unvalidated input reaches a service.

2 Parameterised queries

SQL injection closed off through the ORM or bound parameters — never string building.

3 CORS allow-list

Only approved frontend origins. An open CORS policy makes the API everyone's API.

4 Sanitise user input

Especially anything that reaches a name, alias or SKU field.

5 Correlation IDs in logs

Every request traceable end to end. This is what makes production debuggable.

6 No traces in responses

Internal exception detail stays in the log, not in the JSON the client receives.

Note what is absent: authentication is out of scope for v1. That is a decision, and it must be revisited before this system meets a real customer.

The layering, made visible in the folder tree

Backend — FastAPI

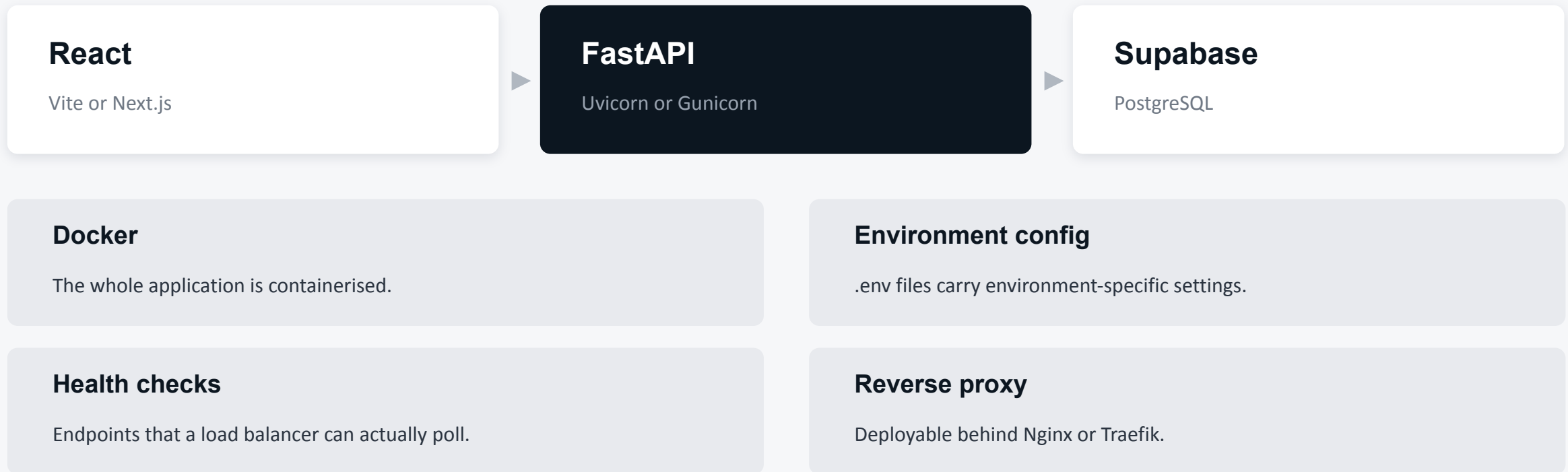
```
app/  
├─ main.py  
├─ config.py  
├─ database.py  
├─ routers/      sessions · inventory · checkout  
├─ services/     session · inventory · checkout · matching  
├─ repositories/ inventory · session  
├─ models/  
├─ schemas/  
├─ middleware/  
└─ utils/
```

Frontend — React

```
src/  
├─ api/  
├─ components/  
├─ hooks/  
├─ pages/        ScanSession.tsx · Inventory.tsx  
├─ services/  
├─ types/  
├─ utils/  
└─ App.tsx
```

Routers hold no logic. Services hold no SQL. Repositories hold no HTTP. If a file breaks that rule, the layering is already gone.

Containerised, configurable, health-checked



Ten statements that decide whether this is done

AC-001 A user can start a new scan session.

AC-002 Products can be scanned and added to the session.

AC-003 Duplicate scans increment quantity, not row count.

AC-004 Matching follows exact, then alias, then fuzzy.

AC-005 Inventory can be created, viewed and updated in the UI.

AC-006 Checkout totals are accurate and run in one transaction.

AC-007 Insufficient stock rejects checkout with no partial updates.

AC-008 Success updates stock, records movements, closes session.

AC-009 All frontend interaction goes through FastAPI REST APIs.

AC-010 Layered, validated, logged, and documented for deployment.

Ten criteria, all binary. None of them requires a meeting to adjudicate.

Everything above was decided before a line was written

01

Scope is a decision, not an outcome

Seven named exclusions mean this release can be estimated, reviewed and finished.

02

Business rules are testable sentences

Six of them. Each maps to an acceptance criterion and, from there, to a test.

03

The architecture constrains the agent

React never reaches the database. That single rule survives every prompt that follows.

This is what a specification looks like when it is written to be executed rather than admired.